Laboratorium 2 - Lambda, Pattern Matching, Higher Order Functions

Spis treści

Instalacja środowiska . . . 2
- Wymagania systemowe . . . 2

Funkcje anonimowe (lambda) . . . 3
- Kiedy używać lambda? . . . 3
- Ograniczenia lambda . . . 3
- Zadania . . . 4

Funkcje wyższego rzędu . . . 5
- Wbudowane funkcje wyższego rzędu . . . 5
- Komponowanie funkcji . . . 6
- Zadania . . . 7

Dekoratory . . . 8
- Podstawowa składnia . . . 8
- Dekoratory z argumentami . . . 8
- Praktyczne przykłady dekoratorów . . . 9
- Łączenie dekoratorów . . . 10
- Zadania . . . 10

Dopasowywanie wzorców (pattern matching) . . . 11
- Podstawowa składnia . . . 11
- Dopasowywanie typów . . . 11
- Dopasowywanie struktur . . . 12
- Dopasowywanie z warunkami (guards) . . . 12
- Dopasowywanie z dataclass . . . 13
- Dopasowywanie list i sekwencji . . . 14
- Zadania . . . 14

Projekt końcowy . . . 16
- System zarządzania hotelem . . . 16

Źródła . . . 18

Celem laboratorium jest poznanie zaawansowanych technik programowania funkcyjnego w Pythonie. Podczas zajęć nauczysz się wykorzystywać funkcje anonimowe (lambda), funkcje wyższego rzędu, dekoratory oraz dopasowywanie wzorców (pattern matching). Na końcu zaimplementujesz system zarządzania rezerwacjami pokojów hotelowych.

Instalacja środowiska

Wymagania systemowe

Do laboratorium potrzebne są:

- Python 3.12 lub nowszy (ze względu na lepsze wsparcie typów)

Funkcje anonimowe (lambda)

Funkcje anonimowe w Pythonie umożliwiają tworzenie bezimiennych metod, które następnie mogą być używane jako wartości. Dodatkowo funkcje lambda zawsze składają się z dokładnie jednego wyrażenia.

```
add = lambda x, y: x + y ①

def add_builder(first_number: int):
    return lambda x: first_number + x ②

print(add(1, 2)) # Result: 3

inc_by_3 = add_builder(3)
print(inc_by_3(2)) # Result: 5
```

① Lambda przyjmuje dwa argumenty: `x` i `y`

② Lambda przyjmuje jeden argument i "zabiera" (*ang. capturing*) `first_number`

Kiedy używać lambda?

Funkcje lambda są szczególnie przydatne w następujących sytuacjach:

```
# Sortowanie z niestandardowym kluczem
students = [("Anna", 85), ("Bob", 92), ("Charlie", 78)]
sorted_students = sorted(students, key=lambda s: s[1], reverse=True) ①
print(sorted_students)  # [('Bob', 92), ('Anna', 85), ('Charlie', 78)]

# Filtrowanie list
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_numbers = list(filter(lambda x: x % 2 == 0, numbers)) ②
print(even_numbers)  # [2, 4, 6, 8, 10]

# Przekształcanie elementów
prices = [10.0, 20.0, 30.0]
prices_with_tax = list(map(lambda p: p * 1.23, prices)) ③
print(prices_with_tax)  # [12.3, 24.6, 36.9]
```

① Sortowanie według drugiego elementu krotki (oceny)

② Filtrowanie - zachowanie tylko liczb parzystych

③ Mapowanie - dodanie podatku 23% do każdej ceny

Ograniczenia lambda

Funkcje lambda mają pewne ograniczenia:

- Mogą zawierać tylko jedno wyrażenie (nie mogą mieć wielu linii)
- Nie mogą zawierać instrukcji jak `if else` (ale mogą wyrazić `if else` w formie wartości)
- Trudniejsze do debugowania (brak nazwy funkcji w stack trace)
- Mogą obniżać czytelność przy złożonej logice

```
# Dobrze - proste wyrażenie
double = lambda x: x * 2

# Dobrze - wyrażenie warunkowe
abs_value = lambda x: x if x >= 0 else -x

# Źle - zbyt skomplikowane, lepiej użyć normalnej funkcji
# complex = lambda x: x * 2 if x > 0 else x * 3 if x < 0 else 0
```

UWAGA: Gdy lambda staje się zbyt skomplikowana, lepiej użyć normalnej funkcji z `def`.

Zadania

1. Napisz funkcję lambda, która przyjmuje listę krotek `(name, age)` i zwraca listę posortowaną według wieku malejąco.
2. Użyj `filter` i lambda do stworzenia listy słów dłuższych niż 5 znaków z listy: `["python", "java", "javascript", "c", "rust"]`
3. Stwórz funkcję `create_multiplier(n)`, która zwraca funkcję lambda mnożącą argument przez `n`.

Funkcje wyższego rzędu

Funkcje wyższego rzędu, to funkcje które jako jeden z swoich argumentów, przyjmują inną funkcję.

```
from typing import Callable

①
def map_fn[T](ls: list[T], fn: Callable[[T], T]) -> list[T]:
    return [fn(el) for el in ls]

ls = [1, 2, 3, 4]
ls = map_fn(ls, lambda x: x * x)
print(ls) # [1, 4, 9, 16]
```

① `T` to typ generyczny

Wbudowane funkcje wyższego rzędu

Python oferuje kilka wbudowanych funkcji wyższego rzędu, które są fundamentem programowania funkcyjnego.

map()

Funkcja `map()` aplikuje daną funkcję do każdego elementu iterowalnej kolekcji:

```
numbers = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x ** 2, numbers)) ①
print(squared)  # [1, 4, 9, 16, 25]

# Z wieloma argumentami
a = [1, 2, 3]
b = [10, 20, 30]
sums = list(
  map(
    lambda x, y: x + y,
    a, b
)) ②
print(sums)  # [11, 22, 33]
```

① Podniesienie każdego elementu do kwadratu

② Mapowanie z dwoma listami jednocześnie

filter()

Funkcja `filter()` wybiera elementy spełniające podany warunek:

```
numbers = range(1, 11)
```

```
even = list(filter(lambda x: x % 2 == 0, numbers)) ①
print(even)  # [2, 4, 6, 8, 10]

words = ["apple", "banana", "kiwi", "strawberry"]
long_words = list(filter(lambda w: len(w) > 5, words)) ②
print(long_words)  # ['banana', 'strawberry']
```

① Filtrowanie liczb parzystych

② Filtrowanie długich słów

reduce()

Funkcja `reduce()` z modułu `functools` redukuje kolekcję do pojedynczej wartości:

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]
sum_all = reduce(lambda acc, x: acc + x, numbers) ①
print(sum_all)  # 15

product = reduce(lambda acc, x: acc * x, numbers, 1) ②
print(product)  # 120
```

① Sumowanie wszystkich elementów

② Iloczyn wszystkich elementów, z wartością początkową 1

Komponowanie funkcji

Funkcje wyższego rzędu można komponować w potoki przetwarzania:

```
from functools import reduce

data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

result = reduce(
    lambda acc, x: acc + x, ①
    map(
        lambda x: x ** 2, ②
        filter(
            lambda x: x % 2 == 0, data ③
        )
    ),
    0
)

print(result) # 220 ④
```

① Sumowanie

② Podniesienie do kwadratu

③ Filtrowanie parzystych

④ $2^2 + 4^2 + 6^2 + 8^2 + 10^2 = 4 + 16 + 36 + 64 + 100$

Zadania

1. Użyj `map()` do przekształcenia listy temperatur w Celsjuszach `[0, 10, 20, 30, 40]` na Fahrenheity (wzór: $F = C \cdot \frac{9}{5} + 32$).
2. Użyj `filter()` i `map()` do stworzenia listy kwadratów liczb nieparzystych z zakresu 1-20.
3. Użyj `reduce()` do znalezienia maksymalnej wartości w liście `[3, 7, 2, 9, 1, 5]`.

Dekoratory

Dekoratory w Pythonie to funkcje wyższego rzędu, które modyfikują zachowanie innych funkcji. Są to funkcje przyjmujące funkcję jako argument i zwracające nową, zmodyfikowaną funkcję.

Podstawowa składnia

```
from typing import Callable

def my_decorator[Y](func: Callable[..., Y]) -> Callable[..., Y]: ①
    def wrapper(*args, **kwargs) -> Y: ②
        print(f"Calling {func.__name__}")
        result = func(*args, **kwargs) ③
        print(f"Finished {func.__name__}")
        return result
    return wrapper

@my_decorator ④
def greet(name: str) -> str:
    return f"Hello, {name}!"

result = greet("Anna")
# Output:
# Calling greet
# Finished greet
```

① Dekorator przyjmuje funkcję jako argument

② Wrapper przyjmuje dowolne argumenty

③ Wywołanie oryginalnej funkcji

④ Użycie dekoratora ze składnią `@`

Dekoratory z argumentami

Możemy tworzyć dekoratory przyjmujące dowolne argumenty (jak każda inna funkcja):

```
def repeat(times: int): ①
    def decorator(func): ②
        def wrapper(*args, **kwargs):
            result = None
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(times=3) ③
```

```
def greet(name: str) -> None:
    print(f"Hello, {name}!")

greet("Anna")
# Output:
# Hello, Anna!
# Hello, Anna!
# Hello, Anna!
```

① Zewnętrzna funkcja przyjmuje argumenty dekoratora

② Wewnętrzna funkcja jest właściwym dekoratorem

③ Wywołanie dekoratora z argumentem

Praktyczne przykłady dekoratorów

Standardowa biblioteka Pythona udostępnia dużo wbudowanych dekoratorów, które można użyć aby zaoszczędzić trochę czasu przy implementacji. Większość z nich zdefiniowana jest w module programowania funkcyjnego Pythona - `functools`.

Poniżej przedstawione jest parę przydatnych przykładów dekoratorów w Pythonie.

Cachowanie wyników

Python oferuje wbudowany dekorator `@lru_cache` do zapamiętywania wyników:

```
from functools import lru_cache

@lru_cache(maxsize=128) ①
def fibonacci(n: int) -> int:
    if n < 2:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(100))  # Obliczone bardzo szybko dzięki cache'owaniu
```

① Cache dla maksymalnie 128 wyników

Walidacja typów

```
def validate_positive(func):
    def wrapper(x: int) -> int:
        if x < 0:
            raise ValueError("Argument must be positive")
        return func(x)
    return wrapper

@validate_positive
```

```python
def calculate_discount(price: int) -> float:
    return price * 0.9

print(calculate_discount(100))  # 90.0
# calculate_discount(-50)  # ValueError: Argument must be positive
```

Łączenie dekoratorów

Dekoratory można komponować na jednej funkcji. Spowoduje to ich wykonanie zgodnie z kolejnością ich wywołań/definicji:

```python
@decorator_one
@decorator_two
@decorator_three
def my_function():
    pass

# Równoważne z:
# my_function = decorator_one(decorator_two(decorator_three(my_function)))
```

Zadania

1. Napisz dekorator `@log_calls`, który wyświetla nazwę funkcji i jej argumenty przed każdym wywołaniem.
2. Stwórz dekorator `@only_positive`, który sprawdza czy wszystkie argumenty numeryczne są dodatnie.

Dopasowywanie wzorców (pattern matching)

Pattern matching (instrukcja `match-case`) został wprowadzony w Python 3.10 i oferuje elegancki sposób na dopasowywanie struktur danych i wykonywanie kodu w zależności od ich kształtu.

Podstawowa składnia

```
def describe_number(n: int) -> str:
    match n: ①
        case 0: ②
            return "zero"
        case 1:
            return "one"
        case 2:
            return "two"
        case _: ③
            return "many"

print(describe_number(1))  # "one"
print(describe_number(42)) # "many"
```

① Instrukcja `match` rozpoczyna blok dopasowania, dla wartości `n`.

② Każdy `case` sprawdza konkretną wartość

③ Podłoga - `_` - to wzorzec domyślny (jak `else`)

Dopasowywanie typów

Pattern matching może sprawdzać typy danych:

```
type CrazyData = int | str | list

def process_data(data: CrazyData) -> str:
    match data:
        case int(x): ①
            return f"Integer: {x}"
        case str(s): ②
            return f"String: {s}"
        case list(items): ③
            return f"List with {len(items)} items"
        case _:
            return "Unknown type"

print(process_data(42))        # "Integer: 42"
print(process_data("hello"))   # "String: hello"
```

```
print(process_data([1, 2, 3])) # "List with 3 items"
```

① Dopasowanie liczby całkowitej

② Dopasowanie ciągu znaków

③ Dopasowanie listy

Dopasowywanie struktur

Najpotężniejsza funkcja pattern matching to dopasowywanie złożonych struktur:

```
from typing import Optional

def describe_point(point: tuple) -> Optional[str]:
    match point:
        case (0, 0): ①
            return "Origin"
        case (0, y): ②
            return f"On Y axis at {y}"
        case (x, 0): ③
            return f"On X axis at {x}"
        case (x, y): ④
            return f"Point at ({x}, {y})"
        case _:
            return None

print(describe_point((0, 0)))  # "Origin"
print(describe_point((0, 5)))  # "On Y axis at 5"
print(describe_point((3, 4)))  # "Point at (3, 4)"
```

① Dopasowanie konkretnych wartości

② Dopasowanie z przechwyceniem zmiennej `y`

③ Dopasowanie z przechwyceniem zmiennej `x`

④ Dopasowanie dowolnego punktu

Dopasowywanie z warunkami (guards)

Można dodać dodatkowe warunki do wzorców:

```
def categorize_age(age: int) -> str:
    match age:
        case x if x < 0: ①
            return "Invalid age"
        case x if x < 13:
            return "Child"
        case x if x < 20:
            return "Teenager"
        case x if x < 65:
            return "Adult"
        case _:
            return "Senior"

print(categorize_age(10))  # "Child"
print(categorize_age(25))  # "Adult"
```

① Klauzula `if` dodaje dodatkowy warunek

Dopasowywanie z dataclass

Pattern matching świetnie współpracuje z dataclasses:

```
from dataclasses import dataclass

@dataclass
class Point:
    x: float
    y: float

AXIS_CENTER = Point(0, 0)

@dataclass
class Circle:
    center: Point
    radius: float

@dataclass
class Rectangle:
    top_left: Point
    width: float
    height: float

def describe_shape(shape):
    match shape:
        case Circle(center=AXIS_CENTER, radius=r): ①
            return f"Circle at origin with radius {r}"
        case Circle(center=Point(x=x, y=y), radius=r): ②
            return f"Circle at ({x}, {y}) with radius {r}"
        case Rectangle(top_left=Point(x=x, y=y), width=w, height=h): ③
```

```
            return f"Rectangle at ({x}, {y}), {w}x{h}"
        case _:
            return "Unknown shape"

circle = Circle(Point(0, 0), 5.0)
print(describe_shape(circle))  # "Circle at origin with radius 5.0"

rect = Rectangle(Point(10, 20), 30.0, 40.0)
print(describe_shape(rect))    # "Rectangle at (10, 20), 30.0x40.0"
```

① Dopasowanie okręgu w początku układu współrzędnych

② Dopasowanie dowolnego okręgu

③ Dopasowanie prostokąta

Dopasowywanie list i sekwencji

```
def analyze_list(items: list) -> str:
    match items:
        case []: ①
            return "Empty list"
        case [x]: ②
            return f"Single item: {x}"
        case [x, y]: ③
            return f"Two items: {x} and {y}"
        case [first, *rest]: ④
            return f"First: {first}, rest: {rest}"

print(analyze_list([]))           # "Empty list"
print(analyze_list([1]))          # "Single item: 1"
print(analyze_list([1, 2]))       # "Two items: 1 and 2"
print(analyze_list([1, 2, 3, 4])) # "First: 1, rest: [2, 3, 4]"
```

① Lista pusta

② Lista z jednym elementem

③ Lista z dwoma elementami

④ Pierwszy element i reszta (operator `*`)

Zadania

1. Napisz funkcję `describe_value(val)`, która używa pattern matching do opisania różnych typów danych:
   - `int` → `"Integer: {wartość}"`
   - `str` → `"String of length {długość}"`
   - `list` (pusta) → `"Empty list"`

- `list` (niepusta) → `"List with {liczba} items"`
- inne → `"Unknown type"`

2. Napisz funkcję `process_event(event)`, która przetwarza wydarzenia (w formie krotek) w interfejsie użytkownika:
   - `("click", x, y)` → `"Mouse clicked at ({x}, {y})"`
   - `("keypress", key)` → `"Key pressed: {key}"`
   - `("scroll", delta)` → `"Scrolled by {delta}"`
   - Inne → `"Unknown event"`

Projekt końcowy

System zarządzania hotelem

W tym projekcie stworzysz system zarządzania hotelem wykorzystując techniki programowania funkcyjnego poznane podczas laboratorium: funkcje lambda, funkcje wyższego rzędu oraz dopasowywanie wzorców.

Wymagania funkcjonalne

System powinien implementować następujące funkcjonalności:

- **Zarządzanie pokojami**: System umie zarządzać pokojami
- **Rezerwacje**: System zarządza rezerwacjami pokojów na określone daty (check-in, check-out)
- **Filtrowanie**: Możliwość filtrowania pokojów i rezerwacji za pomocą funkcji użytkownika (lambda)
- **Walidacja**: Rezerwacje pokojów nie mogą się pokrywać w czasie dla tego samego pokoju
- **Sortowanie**: Użytkownik może sortować pokoje według własnego kryterium (funkcja lambda)
- **Dostępność**: System sprawdza dostępność pokojów w żądanych terminach

Architektura

Projekt składa się z dwóch modułów:

- `hotel.py` - moduł z logiką biznesową (do implementacji przez studenta)
- `client.py` - program testowy (dostarczony przez prowadzącego)

Struktura katalogów:

```
projekt/
 ├──── hotel.py         # Do zaimplementowania
 └──── client.py        # Dostarczony
```

Wskazówki implementacyjne

Funkcje wyższego rzędu

Używaj `filter()`, `map()`, `sorted()` z funkcjami lambda jako argumentami (lub list comprehension).

Immutability (nie-zmienność)

Wszystkie dataclass powinny być niemutowalne (`frozen=True`). Funkcje zwracają nowe obiekty zamiast modyfikować istniejące.

Testowanie

Uruchom dostarczony program testowy:

```
cd projekt
python client.py
```

Program testowy sprawdzi wszystkie funkcje i wyświetli wyniki. Wszystkie testy powinny zakończyć się sukcesem.

Kryteria oceny

- Poprawna definicja wszystkich dataclass (20%)
- Implementacja funkcji zarządzania pokojami (20%)
- Implementacja rezerwacji z walidacją konfliktów (25%)
- Używanie funkcji lambda i funkcji wyższego rzędu (25%)
- Wszystkie testy przechodzą pomyślnie (10%)
- **BONUS**: Pattern matching w funkcjach bonusowych (10% extra)

Źródła

- https://peps.python.org/pep-0636/
- https://docs.python.org/3/reference/compound_stmts.html#match
- https://docs.python.org/3/library/typing.html
- https://docs.python.org/3/library/functools.html